# Problema do Caixeiro Viajante (TSP)

Estudo de Caso 01

**Autor 1 - Ítalo Leal Lana Santos**
**Autor 2 - Italo Rodrigues de Matos Avelar**
**Autor 3 - Nathan Filipe Izidorio Alves**
**Autor 4 - Vitor Hugo Dias Santos**
**Autor 5 - Vitor Gabriel da Silva Pereira**

Universidade Federal de Minas Gerais (UFMG)
Prof. Fátima Machado de Souza Lima

## 1 O Problema

O Problema do Caixeiro Viajante, amplamente conhecido pela sigla em inglês *TSP (Traveling Salesman Problem)*, é um dos desafios mais emblemáticos e estudados no campo da Pesquisa Operacional e da Teoria dos Grafos. A premissa, embora simples em sua descrição, oculta uma complexidade matemática profunda: um caixeiro deve visitar um conjunto de cidades exatamente uma vez, partindo de um ponto de origem e retornando a ele, percorrendo a menor distância total possível Matai et al. (2010).

### 1.1 Histórico e Complexidade

As raízes do problema remontam ao século XVIII, com os estudos de Sir William Rowan Hamilton e Thomas Penyngton Kirkman sobre ciclos hamiltonianos. Contudo, sua formalização matemática moderna deve-se a Karl Menger e, posteriormente, a pesquisadores em Princeton como Hassler Whitney. O TSP é classificado na teoria da complexidade computacional como um problema **NP-Difícil ?**. Isso significa que não se conhece um algoritmo de tempo polinomial capaz de encontrar a solução ótima para qualquer instância; à medida que o número de cidades ($n$) cresce, o número de rotas possíveis aumenta fatorialmente ($(n-1)!/2$ em casos simétricos), o que torna a busca exaustiva inviável mesmo para supercomputadores modernos em instâncias de grande porte.

### 1.2 Definição do Estudo de Caso

No contexto deste trabalho, o problema é aplicado à logística de um laticínio que opera uma fábrica e 20 lojas de varejo para reposição semanal de estoque. O cenário é definido pelas seguintes características:

- **Nós:** O depósito (fábrica), designado como nó 0, e as 20 unidades de varejo dispersas geograficamente.
- **Veículo:** Um único caminhão não capacitado, o que elimina restrições de volume de carga, focando exclusivamente na otimização do trajeto.
- **Objetivo:** Determinar a sequência exata de visitas que minimize a distância euclidiana total percorrida, garantindo o retorno à planta original.

### 1.3 Estrutura Matemática

Para a modelagem, considera-se um grafo direcionado $G = (N, A)$, onde:

- $N = \{0, 1, \ldots, n\}$ representa o conjunto de nós (0 para o depósito e $n = 20$ para os clientes).
- $A = \{(i, j) \in N \times N : i \neq j\}$ representa o conjunto de arcos que conectam as localidades.
- $c_{ij}$ é o parâmetro de custo (distância) associado ao deslocamento entre o nó $i$ e o nó $j$.

O desafio central reside na imposição de restrições que impeçam a formação de *subciclos* — rotas fechadas que não incluem o depósito, o que exige uma formulação matemática sofisticada para garantir a conectividade de toda a rota.

## 2 Formulação do Modelo Matemático Generalizado

Para a formalização do Problema do Caixeiro Viajante (TSP) com as restrições de eliminação de subciclos propostas por Gavish e Graves (1978), as regras de roteamento e conectividade logística são traduzidas em expressões algébricas. O modelo generalizado abaixo apresenta a estrutura matemática necessária para determinar a rota que minimize a distância total percorrida, garantindo que todos os nós sejam visitados exatamente uma vez sem a formação de rotas desconectadas (sub-rotas).

Para a modelagem, consideramos o problema definido em um grafo direcionado $G = (N, A)$.

### 2.1 Conjuntos

- $N = \{0, 1, \ldots, n\}$: Conjunto de nós (pontos geográficos), onde 0 representa o depósito (planta/origem) e os demais nós de 1 a $n$ representam os pontos de destino (clientes).
- $A = \{(i, j) : i, j \in N \wedge i \neq j\}$: Conjunto de todos os arcos direcionados válidos que conectam o nó $i$ ao nó $j$.

### 2.2 Parâmetros

- $n$: Quantidade total de clientes (pontos de destino) a serem visitados.
- $c_{ij}$: Custo unitário (representando distância ou tempo de viagem) associado ao deslocamento do veículo do nó $i$ diretamente para o nó $j$, $\forall (i, j) \in A$.

### 2.3 Variáveis de Decisão

O modelo utiliza dois tipos de variáveis de decisão para mapear a rota e garantir a conectividade:

- $x_{ij}$: Variável binária de roteamento. Assume o valor 1 se o arco direcionado de $i$ para $j$ fizer parte da rota ótima, e 0 caso contrário.
- $f_{ij}$: Variável contínua e não-negativa de fluxo. Representa a quantidade de "produto fictício"que o veículo transporta ao longo do arco $(i, j) \in A$.

### 2.4 Função Objetivo

O objetivo primordial do modelo é a minimização do custo total de transporte (neste contexto, a distância total percorrida). Matematicamente, ela é expressa pela soma dos produtos entre a variável de ativação da rota e o respectivo custo do arco:

$$\min Z = \sum_{(i,j) \in A} c_{ij} x_{ij} \tag{1}$$

### 2.5 Restrições

Para que a solução seja viável e represente um único ciclo contínuo (rota hamiltoniana), o modelo deve respeitar as seguintes condições lógicas e físicas:

**a) Restrições de Grau (Entrada e Saída)**

Garantem que o veículo saia exatamente uma única vez de cada nó e chegue exatamente uma única vez em cada nó:

$$\sum_{j:(i,j)\in A} x_{ij} = 1 \quad \forall i \in N \tag{2}$$

$$\sum_{i:(i,j)\in A} x_{ij} = 1 \quad \forall j \in N \tag{3}$$

**b) Restrições de Eliminação de Subciclos (Fluxo Fictício)**

Asseguram que não existam rotas isoladas que não passem pelo depósito. O veículo sai da origem carregando $n$ unidades de carga e descarrega 1 unidade em cada cliente visitado:

- *Geração de fluxo na origem (Depósito 0):*

$$\sum_{(0,j)\in A} f_{0j} = n \tag{4}$$

- *Balanço de fluxo nos clientes (Consumo de 1 unidade):* O fluxo que entra em um cliente $j$ menos o fluxo que sai dele deve ser estritamente igual a 1.

$$\sum_{(i,j)\in A} f_{ij} - \sum_{(j,i)\in A} f_{ji} = 1 \quad \forall j \in N \setminus \{0\} \tag{5}$$

- *Ativação e Capacidade de Fluxo:* O fluxo fictício só pode circular por um arco se o caixeiro realmente viajar por ele ($x_{ij} = 1$). A capacidade máxima do arco é delimitada pelo número total de clientes.

$$f_{ij} \leq n \cdot x_{ij} \quad \forall (i,j) \in A \tag{6}$$

**c) Restrições de Domínio**

Definem a natureza matemática das variáveis do modelo:

$$x_{ij} \in \{0,1\} \quad \forall (i,j) \in A \tag{7}$$

$$f_{ij} \geq 0 \quad \forall (i,j) \in A \tag{8}$$

## 3 Discussão dos resultados

Nesta seção, avaliamos o desempenho do solver `CBC` na resolução do modelo matemático implementado em linguagem Python (utilizando a biblioteca `mip`). Foram executadas três instâncias com dimensões distintas para analisar o comportamento da formulação exata e a escalabilidade do algoritmo frente à complexidade *NP-Difícil* do problema.

### 3.1 Instância 01

1) Tamanho da instância: 21 nós (1 Planta e 20 clientes).

2) Instância: A distribuição espacial das coordenadas baseadas no estudo de caso original está ilustrada na Figura 1.

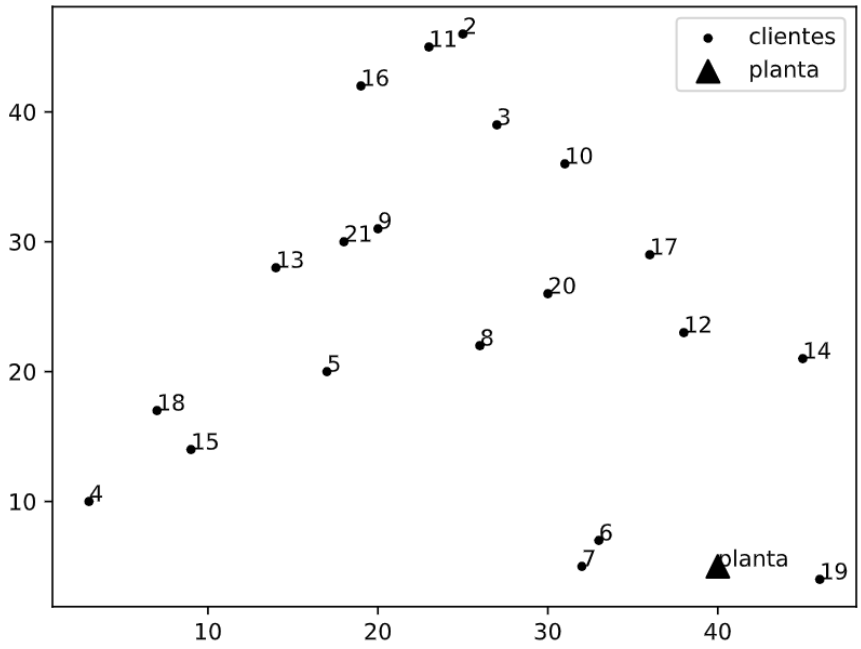


Figura 1: Distribuição dos 21 nós (Instância 01).

3) Resultados: O solver convergiu para a otimalidade global com um tempo de execução de **7.26 segundos**. O custo total mínimo (distância) da rota foi de **165.06**. A rota percorrida gerada pelo modelo está graficada na Figura 2.

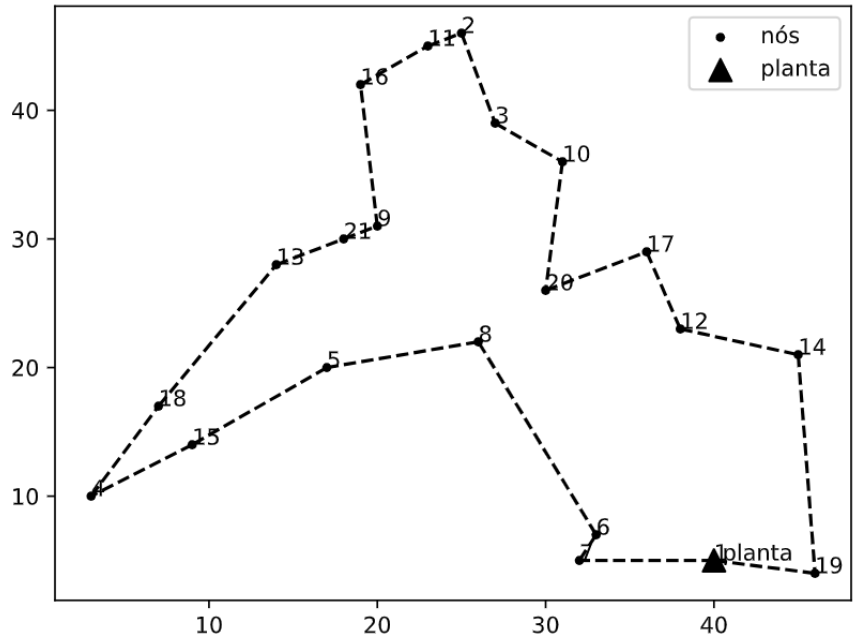


Figura 2: Rota ótima encontrada para a Instância 01.

A sequência de visitas estabelecida sem a formação de subciclos foi: Planta (1) $\rightarrow$ 7 $\rightarrow$ 6 $\rightarrow$ 8 $\rightarrow$ 5 $\rightarrow$ 15 $\rightarrow$ 4 $\rightarrow$ 18 $\rightarrow$ 13 $\rightarrow$ 21 $\rightarrow$ 9 $\rightarrow$ 16 $\rightarrow$ 11 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 17 $\rightarrow$ 12 $\rightarrow$ 14 $\rightarrow$ 19 $\rightarrow$ Planta (1).

### 3.2 Instância 02

1) Tamanho da instância: 34 nós (1 Planta e 33 clientes).

2) Instância: Gerada para testar o estresse combinatório do solver, espalhando mais pontos pela malha bidimensional (Figura 3).

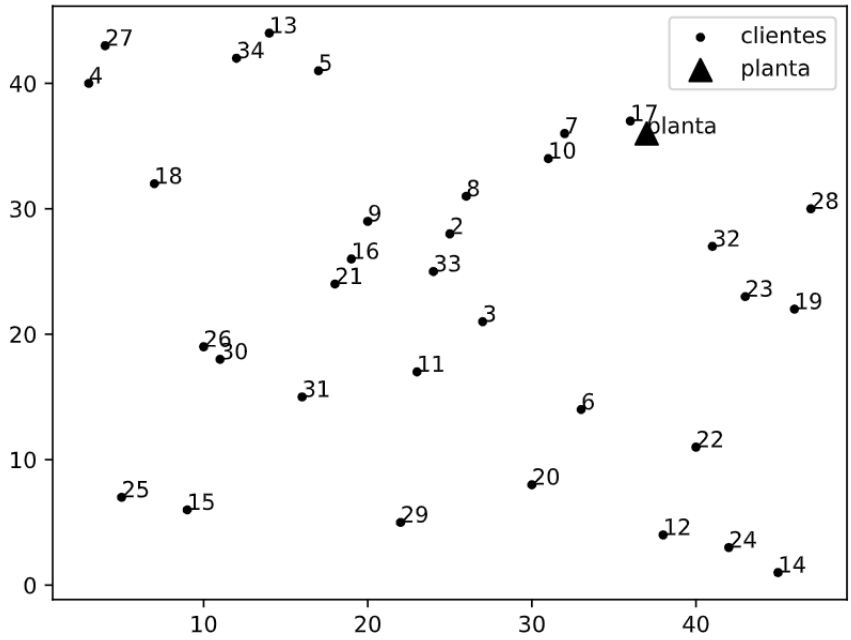


Figura 3: Distribuição dos 34 nós (Instância 02).

3) Resultados: O acréscimo de 13 clientes multiplicou a complexidade computacional do modelo. O tempo de execução saltou para **81.77 segundos**, exigindo que o solver avaliasse milhares de iterações e nós na árvore de *branch-and-bound*. O custo total da rota ótima foi de **219.42**, ilustrada na Figura 4.

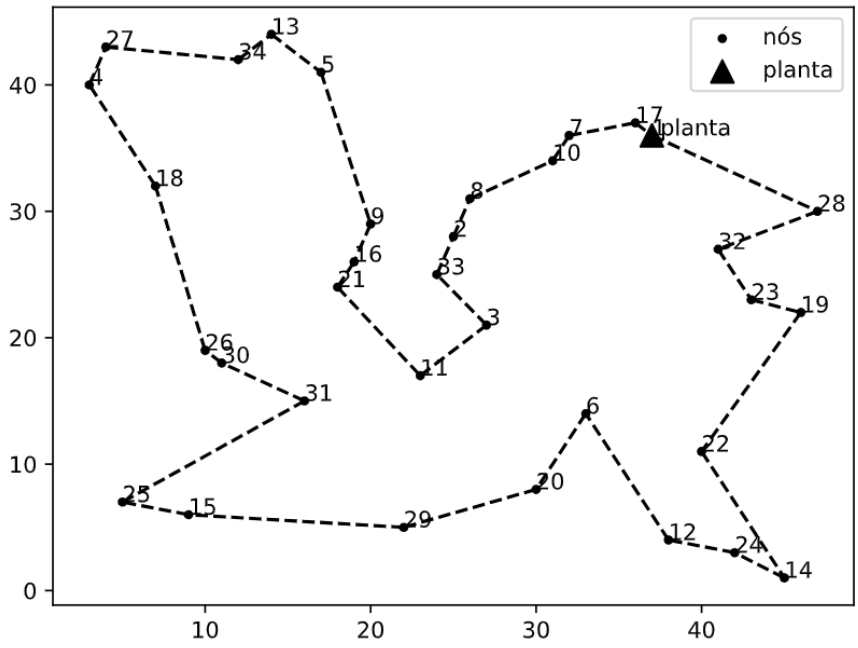


Figura 4: Rota ótima encontrada para a Instância 02.

A sequência ótima gerada foi: Planta (1) $\rightarrow$ 28 $\rightarrow$ 32 $\rightarrow$ 23 $\rightarrow$ 19 $\rightarrow$ 22 $\rightarrow$ 14 $\rightarrow$ 24 $\rightarrow$ 12 $\rightarrow$ 6 $\rightarrow$ 20 $\rightarrow$ 29 $\rightarrow$ 15 $\rightarrow$ 25 $\rightarrow$ 31 $\rightarrow$ 30 $\rightarrow$ 26 $\rightarrow$ 18 $\rightarrow$ 4 $\rightarrow$ 27 $\rightarrow$ 34 $\rightarrow$ 13 $\rightarrow$ 5 $\rightarrow$ 9 $\rightarrow$ 16 $\rightarrow$ 21 $\rightarrow$ 11 $\rightarrow$ 3 $\rightarrow$ 33 $\rightarrow$ 2 $\rightarrow$ 8 $\rightarrow$ 10 $\rightarrow$ 7 $\rightarrow$ 17 $\rightarrow$ Planta (1).

### 3.3 Instância 03

1) Tamanho da instância: 14 nós (1 Planta e 13 clientes).

2) Instância: Uma configuração de menor porte gerada para contrastar o impacto de redução no tempo de processamento. A disposição geográfica está apresentada na Figura 5.

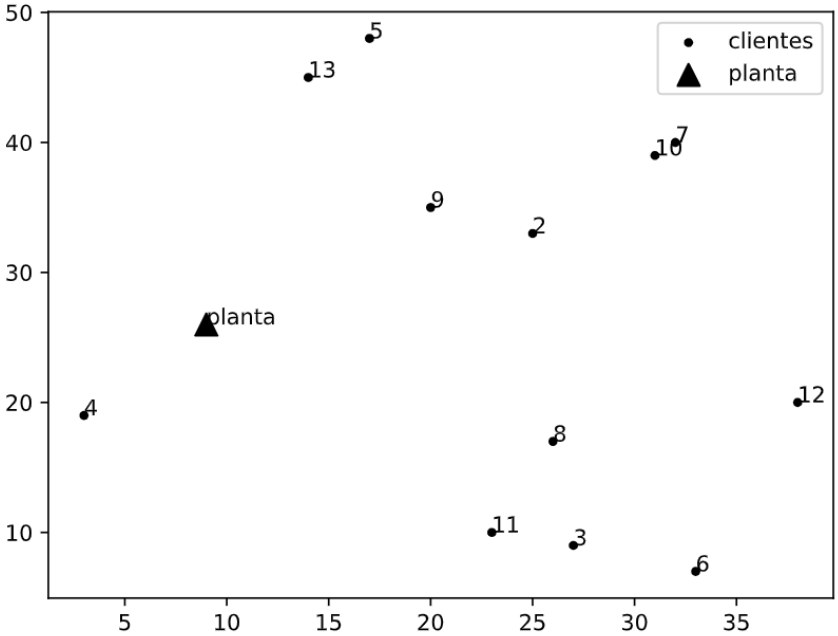


Figura 5: Distribuição dos 14 nós (Instância 03).

3) Resultados: Evidenciando a relação exponencial entre o número de variáveis e o esforço computacional no TSP, esta instância de menor escala foi resolvida quase instantaneamente. O tempo total de execução foi de apenas **1.32 segundos**, atingindo um custo mínimo ótimo de **137.69**. A solução visual encontra-se na Figura 6.

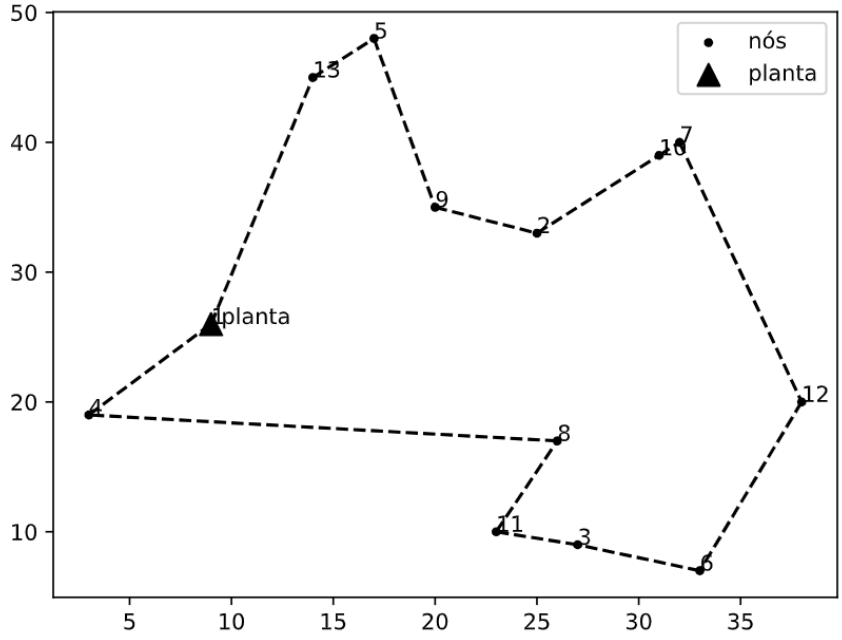


Figura 6: Rota ótima encontrada para a Instância 03.

A rota final estabelecida, visitando cada ponto exatamente uma vez, foi: Planta (1) $\rightarrow$ 13 $\rightarrow$ 5 $\rightarrow$ 9 $\rightarrow$ 2 $\rightarrow$ 10 $\rightarrow$ 7 $\rightarrow$ 12 $\rightarrow$ 6 $\rightarrow$ 3 $\rightarrow$ 11 $\rightarrow$ 8 $\rightarrow$ 4 $\rightarrow$ Planta (1).

# 4 Aplicações práticas do TSP

O Problema do Caixeiro Viajante (TSP) possui ampla aplicação em diversos contextos reais que envolvem otimização de rotas e minimização de custos. A seguir, são apresentadas cinco aplicações práticas relevantes, com base na literatura científica.

## 4.1 1. Roteamento de Veículos de Entrega

Uma das aplicações mais clássicas do TSP está na logística de distribuição de mercadorias. Empresas utilizam esse modelo para definir a melhor rota de entrega, minimizando distância percorrida, tempo e consumo de combustível. Nesse contexto:

- O depósito representa o ponto de origem (nó 0);
- Os clientes são os pontos de entrega;
- O custo $c_{ij}$ representa a distância ou tempo entre locais.

Segundo Matai et al. (2010), essa aplicação é uma das mais estudadas devido ao seu impacto direto na redução de custos operacionais.

## 4.2 2. Coleta de Resíduos Urbanos

O TSP também é aplicado na definição de rotas para coleta de lixo em cidades. O objetivo é percorrer todos os pontos de coleta com o menor custo possível.

Neste caso:

- O depósito é o centro de descarte;

- Os nós representam pontos de coleta;
- A otimização reduz tempo de operação e emissão de poluentes.

Essa aplicação é amplamente discutida em problemas de logística urbana e sustentabilidade.

### 4.3 3. Planejamento de Circuitos em Placas Eletrônicas

Na fabricação de placas de circuito impresso (PCBs), o TSP é utilizado para otimizar o caminho de perfuração ou soldagem de componentes.

- Cada ponto representa uma posição na placa;
- O objetivo é minimizar o deslocamento da máquina;
- Isso reduz tempo de produção e desgaste do equipamento.

Esse problema é conhecido como *drilling problem* na literatura.

### 4.4 4. Sequenciamento de DNA

Na bioinformática, o TSP é utilizado em problemas de montagem de sequências genéticas.

- Cada fragmento de DNA é tratado como um nó;
- O custo representa a similaridade entre fragmentos;
- O objetivo é encontrar a melhor ordem de reconstrução.

Embora mais complexo, esse tipo de aplicação demonstra a versatilidade do modelo.

### 4.5 5. Planejamento de Rotas para Drones

Com o avanço da tecnologia, o TSP passou a ser aplicado no planejamento de rotas para drones de entrega e monitoramento.

- Os pontos representam locais de entrega ou inspeção;
- O custo pode incluir distância, tempo ou consumo de bateria;
- A otimização aumenta a autonomia e eficiência operacional.

Essa é uma aplicação moderna e crescente, especialmente em sistemas logísticos automatizados.

### 4.6 Relação com o Modelo Matemático

Em todas essas aplicações, o problema pode ser modelado da mesma forma apresentada neste trabalho:

- Variáveis binárias $x_{ij}$ definem a rota;
- A função objetivo minimiza o custo total;
- As restrições garantem visita única e eliminam subciclos.

Dessa forma, o modelo matemático do TSP serve como base para resolver problemas reais de diferentes naturezas, demonstrando sua importância na Pesquisa Operacional.

# 5 Código do modelo em Python

A seguir, são apresentados os scripts desenvolvidos em linguagem Python para a simulação e resolução do estudo de caso. O primeiro script foi utilizado para a geração das coordenadas geográficas aleatórias das instâncias testadas. O segundo script contém a declaração completa do modelo matemático, a configuração das restrições de subciclos utilizando a biblioteca `mip`, a chamada para o solver `CBC` e a rotina de plotagem gráfica.

## 5.1 Gerador de Instâncias (`new_instances.py`)

```
from random import seed, randint, sample
import matplotlib.pyplot as plt
import numpy as np

seed(0)

# ni,nj = 33,1 # Instância 1
# ni,nj = 20,1 # Instância 2
ni,nj = 12, 1 # Instância 3
I,J = range(ni),range(nj)

coordxi = sample(range(1,50),ni)
coordyi = sample(range(1,50),ni)
coordxj = sample(range(5,45),nj)
coordyj = sample(range(5,45),nj)

fig, ax = plt.subplots()
plt.scatter(coordxi,coordyi,marker="o",color='black',s=10,label="clientes")
for i in I:
    plt.text(coordxi[i],coordyi[i], "{:d}".format(i + 2))

plt.scatter(coordxj,coordyj,marker="^",color='black',s=100,label="planta")
for j in J:
    plt.text(coordxj[j],coordyj[j], "{:s}".format("planta"))
    plt.legend()
    plt.plot()
    plt.savefig("exemplo_caso_teste.pdf")
    #plt.show()

dj = [ [1,coordxj[j],coordyj[j]] for j in J]
di = [ [i+2,coordxi[i],coordyi[i]] for i in I]
dados = np.array(dj + di)
print("param : posx posy :=")

for (i,x,y) in dados:
    print(" {:3d} {:12.2f} {:12.2f}".format(i,x,y))
    print(";")
    print()
    labels = ['posx', 'posy']
    for h in range(1,3):
        print("{:s} = np.array([".format(labels[h-1]),end='')
```

```
        for i,v in enumerate(dados[:,h]):
            print("{:d}".format(v),end='')
            if i == len(dados) - 1:
                print("])".format(v))
            else:
                print(", ".format(v),end='')
```

### 5.2 Modelo de Otimização TSP (tsp.py)

```
from mip import Model, xsum, minimize, CBC, OptimizationStatus, BINARY
from itertools import product
import matplotlib.pyplot as plt
from math import sqrt
import numpy as np

# numero de clientes,no de identificacao da planta
# n = 34 # Instância 1
# n = 21 # Instância 2
n = 13 # Instância 3
planta = 0

# listas com os indices dos clientes e dos arcos
N = range(n)
A = [(i,j) for (i,j) in product(N,N) if i != j]

# posicao geografica dos nos

# Instância 1
# posx = np.array([37, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14, 45, 9, 19,
#                  36, 7, 46, 30, 18, 40, 43, 42, 5, 10, 4, 47, 22, 11, 16, 41, 24, 12])
# posy = np.array([36, 28, 21, 40, 41, 14, 36, 31, 29, 34, 17, 4, 44, 1, 6, 26,
#                  37, 32, 22, 8, 24, 11, 23, 3, 7, 19, 43, 30, 5, 18, 15, 27, 25, 42])

# Instância 2
# posx = np.array([40, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14, 45, 9, 19,
#                  36, 7, 46, 30, 18])
# posy = np.array([5, 46, 39, 10, 20, 7, 5, 22, 31, 36, 45, 23, 28, 21, 14, 42,
#                  29, 17, 4, 26, 30])

# Instância 3
posx = np.array([9, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14])
posy = np.array([26, 33, 9, 19, 48, 7, 40, 17, 35, 39, 10, 20, 45])

# distancia Euclidiana entre os nos
c = [ [ sqrt( (posx[i] - posx[j])**2 + (posy[i] - posy[j])**2 ) for j in N ] for i in N]

# declaracao do modelo
model = Model('Problema do caixeiro viajante',solver_name=CBC)

# x_ij igual a 1 se no j e visitado imediatamente apos no i; 0, caso contrario
x = {(i,j) : model.add_var(var_type=BINARY) for (i,j) in A}
```

```
# f_ij qtde de produto ficticio transportada pelo caixeiro viajante no arco (i,j)
f = {(i,j) : model.add_var(lb=0.0) for (i,j) in A}

# funcao objetvio: minimizacao do comprimento/distancia da rota
model.objective = minimize(xsum(c[i][j] * x[i,j] for (i,j) in A))

# restricoes do grau de cada no: em t0do no, so chega um arco e so sai um arco dele
for i in N:
    model += xsum(x[ii,j] for (ii,j) in A if i == ii) == 1
for j in N:
    model += xsum(x[i,jj] for (i,jj) in A if j == jj) == 1

# restricoes para eliminacao de subciclos ou subrotas
# o caixeiro sai da planta com n-1 unidades de produto ficticio
# para deixar uma unidade em cada no a ser visitado
#restricoes de ativacao: um cliente i so pode ser atendido por j se j estiver instalado
model += xsum(f[i,j] for (i,j) in A if i == planta) == n - 1

# restricao de balanco de fluxo: o caixeiro chega em um no com uma determinada
# quantidade de produto ficticio, deixa uma unidade deste produto, e sai
# deste no carregando uma unidade a menos.
for j in N:
    if j != planta:
        model += xsum(f[i,jj] for (i,jj) in A if j == jj) == 1 + xsum(f[jj,i] for (jj,i) in A if

# restricao de ativacao do arco: o caixeiro so pode usar o arco (i,j)
# para carregar produto ficticio se o arco for contabilizado na funcao objetivo
for (i,j) in A:
    model += f[i,j] <= (n-1) * x[i,j]

# otimiza o modelo chamando o resolvedor
status = model.optimize()
if status == OptimizationStatus.OPTIMAL:
    print("Custo da rota: {:12.2f}".format(sum([c[i][j] * x[i,j].x for (i,j) in A])))
    cur_no = planta
    print("Rota: ")
    while True:
        print("{:3d} ".format(cur_no+1),end='')
        cur_no = [j for (i,j) in A if i == cur_no and x[i,j].x > 0.9][0]
        if cur_no == planta:
            break;
    print(), print()

    fig, ax = plt.subplots()
    plt.scatter(posx[:],posy[:],marker="o",color='black',s=10,label="nós")
    for i in N:
        plt.text(posx[i],posy[i], "{:d}".format(i + 1))

    for (i, j) in [(i, j) for (i, j) in A if x[i, j].x >= .9]:
        plt.plot((posx[i], posx[j]), (posy[i], posy[j]), linestyle="--", color="black")
```

```
plt.scatter(posx[0],posy[0],marker="^",color='black',s=100,label="planta")
plt.text(posx[0]+.5,posy[0], "{:s}".format("planta"))
plt.legend()
plt.plot()
plt.savefig("exemplo_solucao.pdf")
plt.show()
```

## Referências


Matai, R., Singh, S. P., & Mittal, M. L. (2010). Traveling salesman problem: an overview of applications, formulations, and solution approaches. *Traveling Salesman Problem, Theory and Applications*, *1*(1), 1–25.